

Java: Objects and Classes



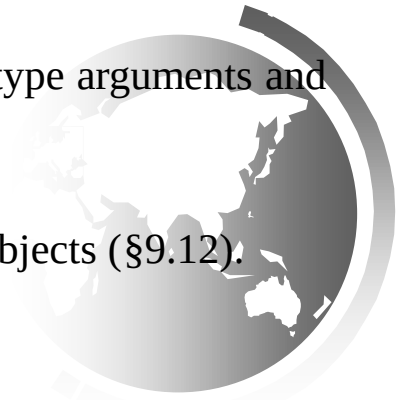
Motivations

After learning the preceding chapters, you are capable of solving many programming problems using selections, loops, methods, and arrays. However, these Java features are not sufficient for developing graphical user interfaces and large scale software systems. In this chapter, we will start discussing object-oriented programming concepts.



Objectives

- ❑ To describe objects and classes, and use classes to model objects (§9.2).
- ❑ To use UML graphical notation to describe classes and objects (§9.2).
- ❑ To demonstrate how to define classes and create objects (§9.3).
- ❑ To create objects using constructors (§9.4).
- ❑ To access objects via object reference variables (§9.5).
- ❑ To define a reference variable using a reference type (§9.5.1).
- ❑ To access an object's data and methods using the object member access operator (.) (§9.5.2).
- ❑ To define data fields of reference types and assign default values for an object's data fields (§9.5.3).
- ❑ To distinguish between object reference variables and primitive data type variables (§9.5.4).
- ❑ To use the Java library classes **Date**, **Random**, and **Point2D** (§9.6).
- ❑ To distinguish between instance and static variables and methods (§9.7).
- ❑ To define private data fields with appropriate **get** and **set** methods (§9.8).
- ❑ To encapsulate data fields to make classes easy to maintain (§9.9).
- ❑ To develop methods with object arguments and differentiate between primitive-type arguments and object-type arguments (§9.10).
- ❑ To store and process objects in arrays (§9.11).
- ❑ To create immutable objects from immutable classes to protect the contents of objects (§9.12).
- ❑ To determine the scope of variables in the context of a class (§9.13).
- ❑ To use the keyword **this** to refer to the calling object itself (§9.14).



OO Programming Concepts

- Object-oriented programming (OOP) involves programming using objects. An *object* represents an entity in the real world that can be distinctly identified. For example, a student, a desk, a circle, a button, and even a loan can all be viewed as objects.



OO Programming Concepts

- The *state* of an object (also known as its *properties* or *attributes*) is represented by *data fields* with their current values. A circle object, for example, has a data field **radius**, which is the property that characterizes a circle. A rectangle object has the data fields **width** and **height**, which are the properties that characterize a rectangle
- The *behavior* of an object (also known as its *actions*) is defined by methods. To invoke a method on an object is to ask the object to perform an action. For example, you may define methods named **getArea()** and **getPerimeter()** for circle objects. A circle object may invoke **getArea()** to return its area and **getPerimeter()** to return its perimeter.

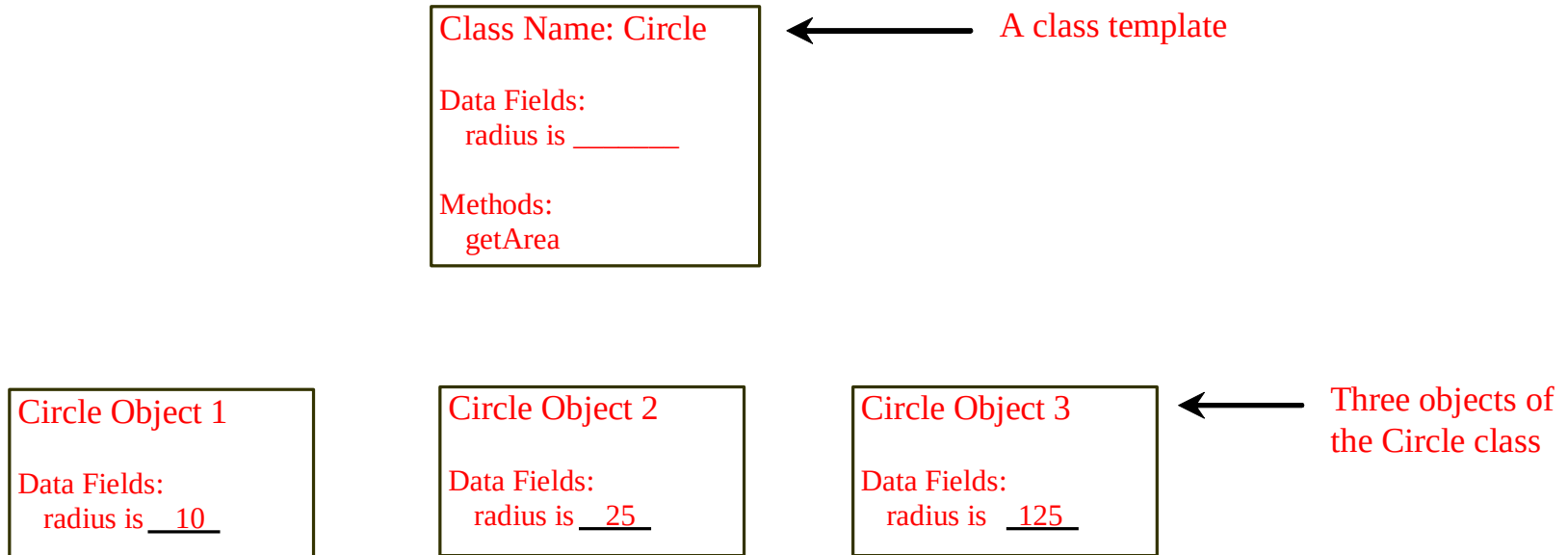


OO Programming Concepts

- Objects of the same type are defined using a common class. A *class* is a template, blueprint, or *contract* that defines what an object's data fields and methods will be. **An object is an instance of a class.** You can create many instances of a class. Creating an instance is referred to as *instantiation*.
- The terms *object* and *instance* are often interchangeable. The relationship between classes and objects is analogous to that between an apple-pie recipe and apple pies: You can make as many apple pies as you want from a single recipe.



Objects



An object has both a state and behavior. The state defines the object, and the behavior defines what the object does.

Classes

Classes are constructs that define objects of the same type. A Java class uses variables to define data fields and methods to define behaviors. Additionally, a class provides a special type of methods, known as **constructors**, which are invoked to construct objects from the class.



Classes

```
class Circle {  
    /** The radius of this circle */  
    double radius = 1.0;  
  
    /** Construct a circle object */  
    Circle() {  
    }  
  
    /** Construct a circle object */  
    Circle(double newRadius) {  
        radius = newRadius;  
    }  
  
    /** Return the area of this circle */  
    double getArea() {  
        return radius * radius * 3.14159;  
    }  
}
```

← Data field

← Constructors

← Method



Classes

- The **Circle** class is different from all of the other classes you have seen thus far. It does not have a **main** method and therefore cannot be run; it is merely a definition for circle objects. The class that contains the **main** method will be referred to in this book, for convenience, as the *main class*.
- The illustration of class templates and objects in can be standardized using *Unified Modeling Language (UML)* notation. This notation is called a *UML class diagram*, or simply a *class diagram*.



Unified Modeling Language (UML) Class Diagram

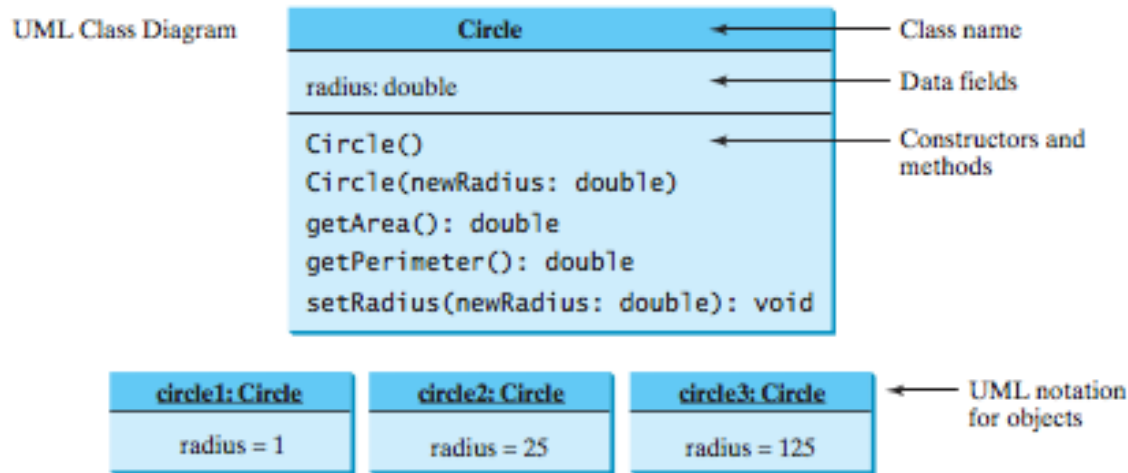


FIGURE 9.4 Classes and objects can be represented using UML notation.

In the class diagram, the data field is denoted as
dataFieldName: dataType

The constructor is denoted as:
ClassName(parameterName: parameterType)

The method is denoted as
methodName(parameterName: parameterType): returnType



Example: *SimpleCircle* class

```
25
26 // Define the circle class with two constructors
27 class SimpleCircle {
28     double radius;
29
30     /** Construct a circle with radius 1 */
31     SimpleCircle() {
32         radius = 1;
33     }
34
35     /** Construct a circle with a specified radius */
36     SimpleCircle(double newRadius) {
37         radius = newRadius;
38     }
39
40     /** Return the area of this circle */
41     double getArea() {
42         return radius * radius * Math.PI;
43     }
44
45     /** Return the perimeter of this circle */
46     double getPerimeter() {
47         return 2 * radius * Math.PI;
48     }
49
50     /** Set a new radius for this circle */
51     void setRadius(double newRadius) {
52         radius = newRadius;
53     }
54 }
```



Example: *SimpleCircle* class

```
1 public class TestSimpleCircle {
2     /** Main method */
3     public static void main(String[] args) {
4         // Create a circle with radius 1
5         SimpleCircle circle1 = new SimpleCircle();
6         System.out.println("The area of the circle of radius "
7             + circle1.radius + " is " + circle1.getArea());
8
9         // Create a circle with radius 25
10        SimpleCircle circle2 = new SimpleCircle(25);
11        System.out.println("The area of the circle of radius "
12            + circle2.radius + " is " + circle2.getArea());
13
14        // Create a circle with radius 125
15        SimpleCircle circle3 = new SimpleCircle(125);
16        System.out.println("The area of the circle of radius "
17            + circle3.radius + " is " + circle3.getArea());
18
19        // Modify circle radius
20        circle2.radius = 100; // or circle2.setRadius(100)
21        System.out.println("The area of the circle of radius "
22            + circle2.radius + " is " + circle2.getArea());
23    }
24 }
```



Example: *SimpleCircle* class

- The program contains two classes. The first of these, **TestSimpleCircle**, is the main class. Its sole purpose is to test the second class, **SimpleCircle**. Such a program that uses the class is often referred to as a *client* of the class. When you run the program, the Java runtime system invokes the **main** method in the main class.
- You can put the two classes into one file, **but only one class in the file can be a *public class***. Furthermore, the public class must have the same name as the file name. Therefore, the file name is **TestSimpleCircle.java**, since **TestSimpleCircle** is public.
- Each class in the source code is compiled into a **.class** file. When you compile **TestSimpleCircle.java**, two class files **TestSimpleCircle.class** and **SimpleCircle.class** are generated,

Combine Two Classes into One

```
1 public class SimpleCircle {
2     /** Main method */
3     public static void main(String[] args) {
4         // Create a circle with radius 1
5         SimpleCircle circle1 = new SimpleCircle();
6         System.out.println("The area of the circle of radius "
7             + circle1.radius + " is " + circle1.getArea());
8
9         // Create a circle with radius 25
10        SimpleCircle circle2 = new SimpleCircle(25);
11        System.out.println("The area of the circle of radius "
12            + circle2.radius + " is " + circle2.getArea());
13
14        // Create a circle with radius 125
15        SimpleCircle circle3 = new SimpleCircle(125);
16        System.out.println("The area of the circle of radius "
17            + circle3.radius + " is " + circle3.getArea());
18
19        // Modify circle radius
20        circle2.radius = 100;
21        System.out.println("The area of the circle of radius "
22            + circle2.radius + " is " + circle2.getArea());
23    }
24
25    double radius;
26
27    /** Construct a circle with radius 1 */
28    SimpleCircle() {
29        radius = 1;
30    }
31
32    /** Construct a circle with a specified radius */
33    SimpleCircle(double newRadius) {
34        radius = newRadius;
35    }
36
37    /** Return the area of this circle */
38    double getArea() {
39        return radius * radius * Math.PI;
40    }
41}
```



Combine Two Classes into One

```
42  /** Return the perimeter of this circle */
43  double getPerimeter() {
44      return 2 * radius * Math.PI;
45  }
46
47  /** Set a new radius for this circle */
48  void setRadius(double newRadius) {
49      radius = newRadius;
50  }
51 }
```



Example: Defining Classes and Creating Objects

TV	
channel: int volumeLevel: int on: boolean	
+TV() +turnOn(): void +turnOff(): void +setChannel(newChannel: int): void +setVolume(newVolumeLevel: int): void +channelUp(): void +channelDown(): void +volumeUp(): void +volumeDown(): void	

The + sign indicates
a public modifier. →

The current channel (1 to 120) of this TV.
The current volume level (1 to 7) of this TV.
Indicates whether this TV is on/off.

Constructs a default TV object.
Turns on this TV.
Turns off this TV.
Sets a new channel for this TV.
Sets a new volume level for this TV.
Increases the channel number by 1.
Decreases the channel number by 1.
Increases the volume level by 1.
Decreases the volume level by 1.

The constructor and methods in the **TV** class are defined public so they can be accessed from other classes.



Example: Defining Classes and Creating Objects

```
public class TV {  
    int channel = 1; // Default channel  
    int volumeLevel = 1; // Default volume level  
    boolean on = false; // TV is off  
  
    public TV() {  
    }  
  
    public void turnOn() {  
        on = true;  
    }  
  
    public void turnOff() {
```

```
14         on = false;  
15     }  
16  
17     public void setChannel(int newChannel) {  
18         if (on && newChannel >= 1 && newChannel <= 120)  
19             channel = newChannel;  
20     }  
21  
22     public void setVolume(int newVolumeLevel) {  
23         if (on && newVolumeLevel >= 1 && newVolumeLevel <= 7)  
24             volumeLevel = newVolumeLevel;  
25     }  
26  
27     public void channelUp() {  
28         if (on && channel < 120)  
29             channel++;  
30     }  
31  
32     public void channelDown() {  
33         if (on && channel > 1)  
34             channel--;  
35     }  
36  
37     public void volumeUp() {  
38         if (on && volumeLevel < 7)  
39             volumeLevel++;  
40     }  
41  
42     public void volumeDown() {  
43         if (on && volumeLevel > 1)  
44             volumeLevel--;  
45     }  
46 }
```

Example: Defining Classes and Creating Objects

```
public class TestTV {  
    public static void main(String[] args) {  
        TV tv1 = new TV();  
        tv1.turnOn();  
        tv1.setChannel(30);  
        tv1.setVolume(3);  
  
        TV tv2 = new TV();  
        tv2.turnOn();  
        tv2.channelUp();  
        tv2.channelUp();  
        tv2.volumeUp();  
  
        System.out.println("tv1's channel is " + tv1.channel  
            + " and volume level is " + tv1.volumeLevel);  
        System.out.println("tv2's channel is " + tv2.channel  
            + " and volume level is " + tv2.volumeLevel);  
    }  
}
```



Constructors

Constructors are a special kind of methods that are invoked to construct objects.

```
Circle() {  
}
```

```
Circle(double newRadius) {  
    radius = newRadius;  
}
```



Constructors, cont.

A constructor with no parameters is referred to as a *no-arg constructor*.

- Constructors must have the same name as the class itself.
- Constructors do not have a return type—not even void.
- Constructors are invoked using the new operator when an object is created. Constructors play the role of initializing objects.



Creating Objects Using Constructors

```
new ClassName() ;
```

Example:

```
new Circle() ;
```

```
new Circle(5.0) ;
```



Default Constructor

A class may be defined without constructors. In this case, a no-arg constructor with an empty body is implicitly defined in the class. This constructor, called *a default constructor*, is provided automatically *only if no constructors are explicitly defined in the class*.



Declaring Object Reference Variables

- ❑ Objects are accessed via the object's *reference variables*, which contain references to the objects.
- ❑ To reference an object, assign the object to a reference variable.
- ❑ A class is a *reference type*, which means that a variable of the class type can reference an instance of the class.
- ❑ To declare a reference variable, use the syntax:

ClassName objectRefVar;

Example:

Circle myCircle;



Declaring/Creating Objects in a Single Step

```
ClassName objectRefVar = new ClassName();
```

Assign object reference

Create an object

Example:

```
Circle myCircle = new Circle();
```



Accessing Object's Members

- ❑ Referencing the object's data is done using the *dot operator* or *object member access operator*:

`objectRefVar.data`

e.g., `myCircle.radius`

- ❑ Invoking the object's method:

`objectRefVar.methodName (arguments)`

e.g., `myCircle.getArea()`



Trace Code

```
Circle myCircle = new Circle(5.0);
```

```
Circle yourCircle = new Circle();
```

```
yourCircle.radius = 100;
```

Declare myCircle

myCircle

no value



Trace Code, cont.

```
Circle myCircle = new Circle(5.0);
```

myCircle no value

```
Circle yourCircle = new Circle();
```

```
yourCircle.radius = 100;
```

<u>: Circle</u>
radius: 5.0

Create a circle



Trace Code, cont.

```
Circle myCircle = new Circle(5.0);
```

```
Circle yourCircle = new Circle();
```

```
yourCircle.radius = 100;
```

Assign object reference
to myCircle

myCircle

reference value

: Circle

radius: 5.0



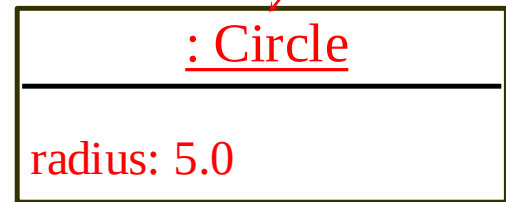
Trace Code, cont.

```
Circle myCircle = new Circle(5.0);
```

```
Circle yourCircle = new Circle();
```

```
yourCircle.radius = 100;
```

myCircle reference value



yourCircle no value

Declare yourCircle

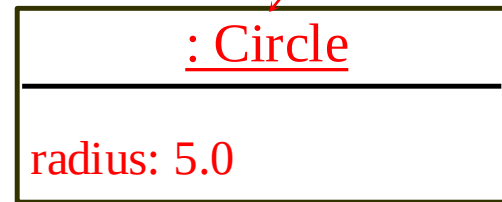
Trace Code, cont.

```
Circle myCircle = new Circle(5.0);
```

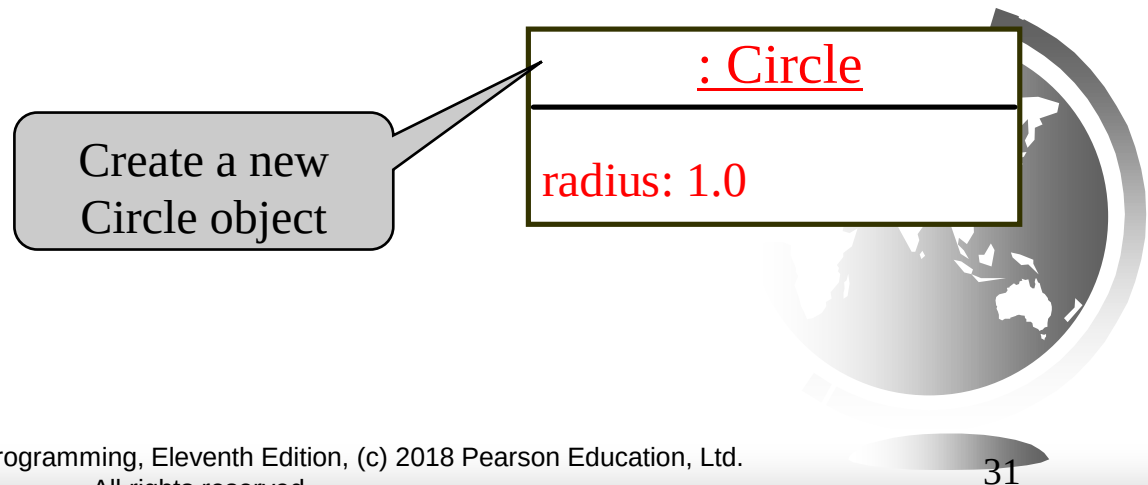
```
Circle yourCircle = new Circle();
```

```
yourCircle.radius = 100;
```

myCircle reference value



yourCircle no value



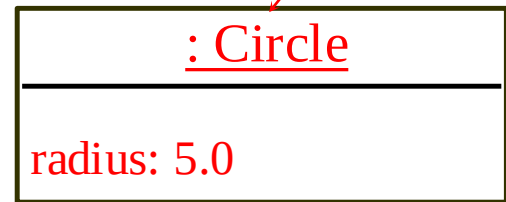
Trace Code, cont.

```
Circle myCircle = new Circle(5.0);
```

```
Circle yourCircle = new Circle();
```

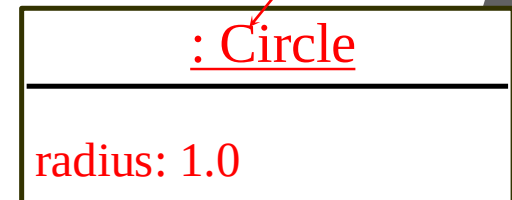
```
yourCircle.radius = 100;
```

myCircle **reference value**



yourCircle **reference value**

Assign object reference
to yourCircle



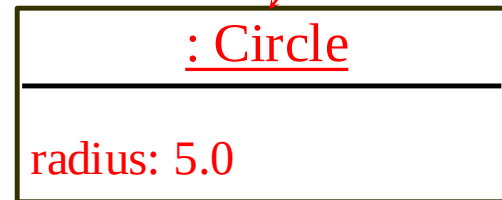
Trace Code, cont.

```
Circle myCircle = new Circle(5.0);
```

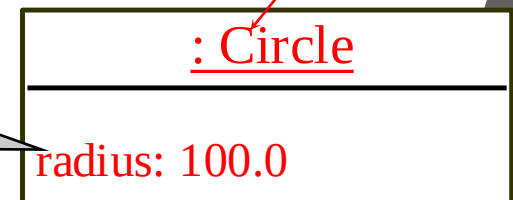
```
Circle yourCircle = new Circle();
```

```
yourCircle.radius = 100;
```

myCircle reference value



yourCircle reference value



Change radius in
yourCircle

Caution

- ❑ The data field **radius** is referred to as an *instance variable*, because it is dependent on a specific instance. For the same reason, the method **getArea** is referred to as an *instance method*, because you can invoke it only on a specific instance.
- ❑ The object on which an instance method is invoked is called a *calling object*.



Caution

- Recall that you use

`Math.methodName(arguments)` (e.g., `Math.pow(3, 2.5)`)

to invoke a method in the `Math` class. Can you invoke `getArea()` using `SimpleCircle.getArea()`? **The answer is no.**

- All the methods used before this chapter are **static methods**, which are defined using the `static` keyword. However, `getArea()` is non-static. It must be invoked from an object using

`objectRefVar.methodName(arguments)` (e.g., `myCircle.getArea()`).



Reference Data Fields

The data fields can be of reference types. For example, the following `Student` class contains a data field `name` of the `String` type.

```
public class Student {  
    String name; // name has default value null  
    int age; // age has default value 0  
    boolean isScienceMajor; // isScienceMajor has default value false  
    char gender; // c has default value '\u0000'  
}
```



The null Value

If a data field of a reference type does not reference any object, the data field holds a special literal value, null.



Default Value for a Data Field

The default value of a data field is null for a reference type, 0 for a numeric type, false for a boolean type, and '\u0000' for a char type.

However, **Java assigns no default value to a local variable inside a method.**

```
public class Test {  
    public static void main(String[] args) {  
        Student student = new Student();  
        System.out.println("name? " + student.name);  
        System.out.println("age? " + student.age);  
        System.out.println("isScienceMajor? " + student.isScienceMajor);  
        System.out.println("gender? " + student.gender);  
    }  
}
```



Example

Java assigns no default value to a local variable inside a method.

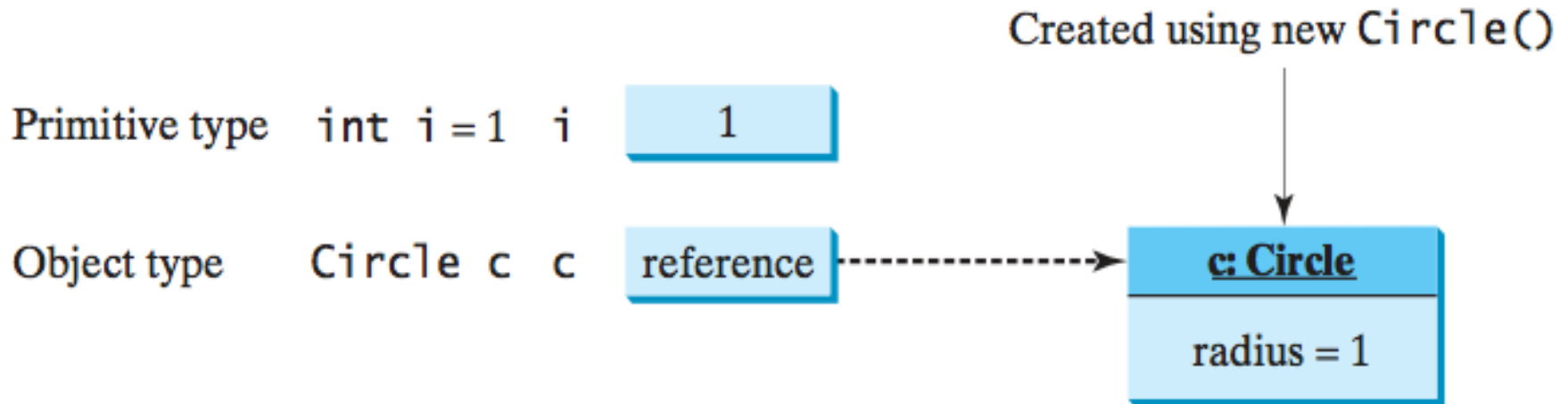
```
public class Test {  
    public static void main(String[] args) {  
        int x; // x has no default value  
        String y; // y has no default value  
        System.out.println("x is " + x);  
        System.out.println("y is " + y);  
    }  
}
```

Compile error: variable not
initialized



Differences between Variables of Primitive Data Types and Object Types

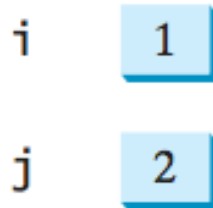
- Every variable represents a memory location that holds a value. When you declare a variable, you are telling the compiler what type of value the variable can hold. For a variable of a primitive type, the value is of the primitive type. For a variable of a reference type, the value is a **reference to where** an object is located.



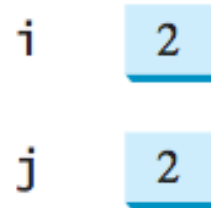
Copying Variables of Primitive Data Types and Object Types

Primitive type assignment $i = j$

Before:

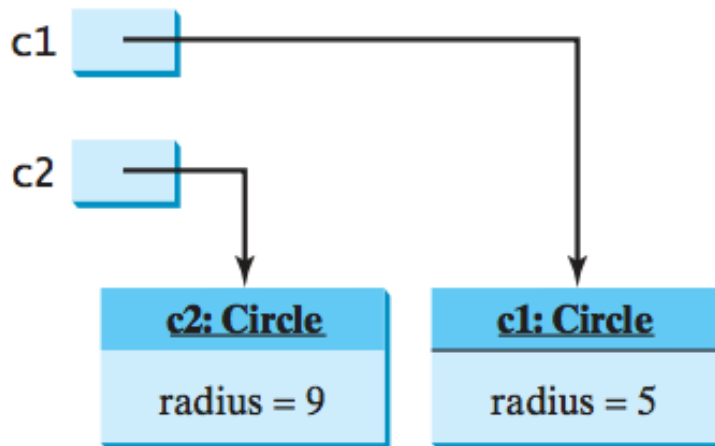


After:

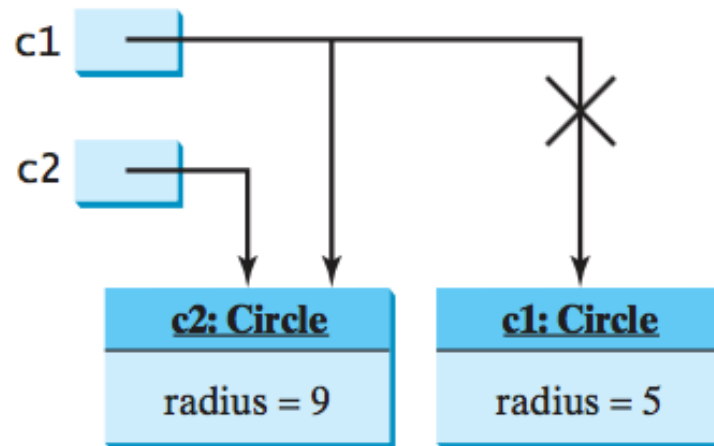


Object type assignment $c1 = c2$

Before:



After:



Garbage Collection

As shown in the previous figure, after the assignment statement `c1 = c2`, `c1` points to the same object referenced by `c2`. The object previously referenced by `c1` is no longer referenced. This object is known as garbage. Garbage is automatically collected by JVM.



Garbage Collection, cont

TIP: If you know that an object is no longer needed, you can explicitly assign null to a reference variable for the object. The JVM will automatically collect the space if the object is not referenced by any variable .



The Date Class

Java provides a system-independent encapsulation of date and time in the `java.util.Date` class. You can use the `Date` class to create an instance for the current date and time and use its `toString` method to return the date and time as a string.

`java.util.Date`

```
+Date()  
+Date(elapseTime: long)  
  
+toString(): String  
+getTime(): long  
  
+setTime(elapseTime: long): void
```

Constructs a `Date` object for the current time.

Constructs a `Date` object for a given time in milliseconds elapsed since January 1, 1970, GMT.

Returns a string representing the date and time.

Returns the number of milliseconds since January 1, 1970, GMT.

Sets a new elapse time in the object.

The Date Class Example

```
java.util.Date date = new java.util.Date();  
System.out.println("The elapsed time since Jan 1, 1970 is " +  
    date.getTime() + " milliseconds");  
System.out.println(date.toString());
```

displays the output like this:

```
The elapsed time since Jan 1, 1970 is 1324903419651 milliseconds  
Mon Dec 26 07:43:39 EST 2011
```



The Random Class

You have used Math.random() to obtain a random double value between 0.0 and 1.0 (excluding 1.0). A more useful random number generator is provided in the java.util.Random class.

java.util.Random	
+Random()	Constructs a Random object with the current time as its seed.
+Random(seed: long)	Constructs a Random object with a specified seed.
+nextInt(): int	Returns a random int value.
+nextInt(n: int): int	Returns a random int value between 0 and n (exclusive).
+nextLong(): long	Returns a random long value.
+nextDouble(): double	Returns a random double value between 0.0 and 1.0 (exclusive).
+nextFloat(): float	Returns a random float value between 0.0F and 1.0F (exclusive).
+nextBoolean(): boolean	Returns a random boolean value.

The Random Class Example

If two Random objects have the same seed, they will generate identical sequences of numbers. For example, the following code creates two Random objects with the same seed 3.

```
Random random1 = new Random(3);  
System.out.print("From random1: ");  
for (int i = 0; i < 10; i++)  
    System.out.print(random1.nextInt(1000) + " ");  
Random random2 = new Random(3);  
System.out.print("\nFrom random2: ");  
for (int i = 0; i < 10; i++)  
    System.out.print(random2.nextInt(1000) + " ");
```

From random1: 734 660 210 581 128 202 549 564 459 961

From random2: 734 660 210 581 128 202 549 564 459 961



The Point2D Class

Java API has a convenient **Point2D** class in the **javafx.geometry** package for representing a point in a two-dimensional plane.

javafx.geometry.Point2D

```
+Point2D(x: double, y: double)
+distance(x: double, y: double): double
+distance(p: Point2D): double
+getX(): double
+getY(): double
+toString(): String
```

Constructs a `Point2D` object with the specified x - and y -coordinates.
Returns the distance between this point and the specified point (x, y) .
Returns the distance between this point and the specified point p .
Returns the x -coordinate from this point.
Returns the y -coordinate from this point.
Returns a string representation for the point.



The Point2D Class

```
import java.util.Scanner;
import javafx.geometry.Point2D;

public class TestPoint2D {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.print("Enter point1's x-, y-coordinates: ");
        double x1 = input.nextDouble();
        double y1 = input.nextDouble();
        System.out.print("Enter point2's x-, y-coordinates: ");
        double x2 = input.nextDouble();
        double y2 = input.nextDouble();

        Point2D p1 = new Point2D(x1, y1);
        Point2D p2 = new Point2D(x2, y2);
        System.out.println("p1 is " + p1.toString());
        System.out.println("p2 is " + p2.toString());
        System.out.println("The distance between p1 and p2 is " +
            p1.distance(p2));
    }
}
```



Instance Variables, and Methods

- Instance variables belong to a specific instance.
- Instance methods are invoked by any instance of the class.



Static Variables, Constants, and Methods

- Static variables are shared by all the instances of the class.
- Static methods are not tied to a specific object. Because of this, a static method cannot access instance members of the class
- Static constants are final variables shared by all the instances of the class.
- A non-static (or instance) variable is tied to a specific instance



Static Variables, Constants, and Methods

- Static variables store values for the variables in a common memory location. Because of this common location, if one object changes the value of a static variable, all objects of the same class are affected.
- Java supports static methods as well as static variables. *Static methods* can be called without creating an instance of the class.



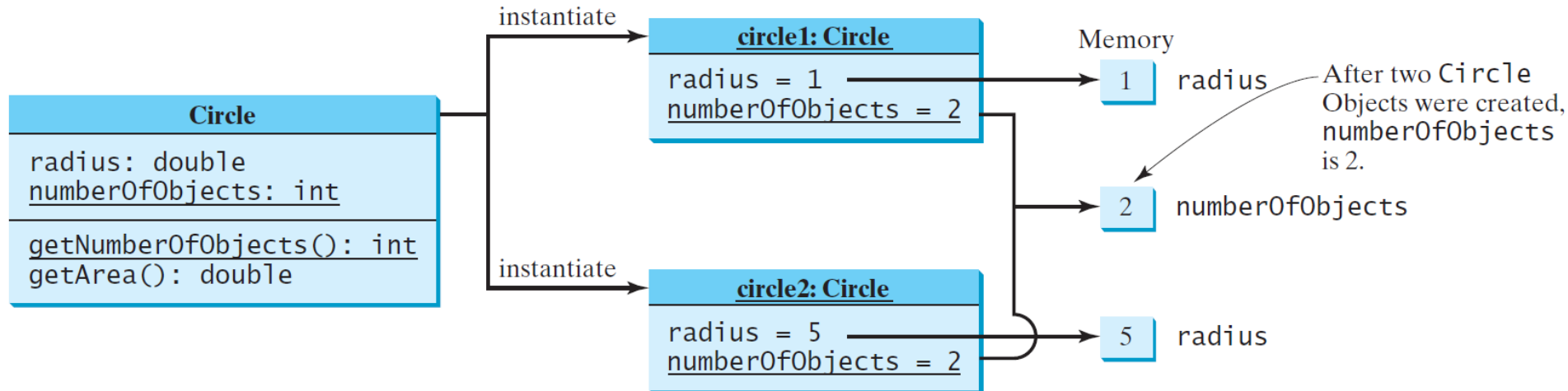
Static Variables, Constants, and Methods, cont.

- To declare static variables, constants, and methods, use the static modifier. For example, the constant **PI** in the **Math** class is defined as
final static double PI=3.14159265358979323846
- Let's modify the **Circle** class by adding a static variable **numberOfObjects** to count the number of circle objects created. When the first object of this class is created, **numberOfObjects** is **1**. When the second object is created, **numberOfObjects** becomes **2**. The UML of the new circle class is shown below

Static Variables, Constants, and Methods, cont.

```
Circle circle1 = new Circle();  
Circle circle2 = new Circle(5);
```

UML Notation:
underline: static variables or methods



CircleWithStaticMembers Class

```
public class CircleWithStaticMembers {  
    /** The radius of the circle */  
    double radius;  
  
    /** The number of objects created */  
    static int numberOfObjects = 0;  
  
    /** Construct a circle with radius 1 */  
    CircleWithStaticMembers() {  
        radius = 1;  
        numberOfObjects++;  
    }  
  
    /** Construct a circle with a specified radius */  
    CircleWithStaticMembers(double newRadius) {  
        radius = newRadius;  
        numberOfObjects++;  
    }  
  
    /** Return numberOfObjects */  
    static int getNumberOfObjects() {  
        return numberOfObjects;  
    }  
  
    /** Return the area of this circle */  
    double getArea() {  
        return radius * radius * Math.PI;  
    }  
}
```



TestCircleWithStaticMembers.java

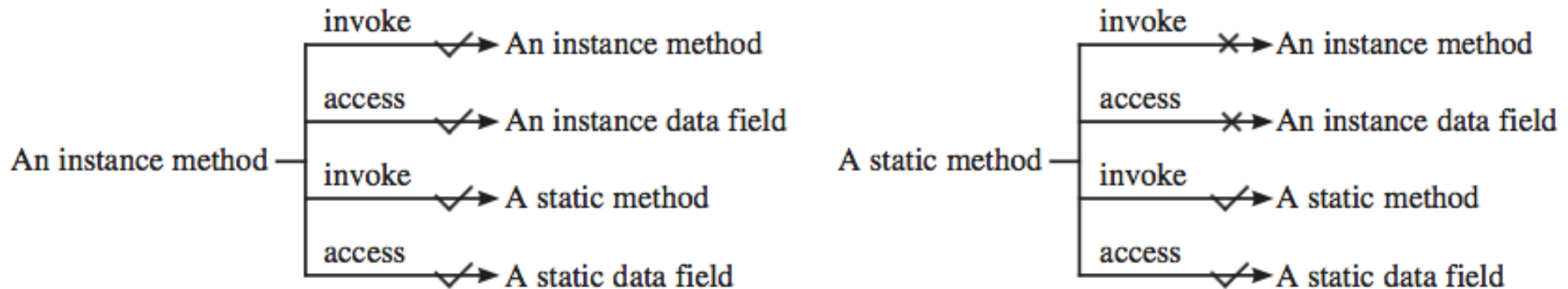
```
public class TestCircleWithStaticMembers {  
    /** Main method */  
    public static void main(String[] args) {  
        System.out.println("Before creating objects");  
        System.out.println("The number of Circle objects is " +  
            CircleWithStaticMembers.numberOfObjects);  
  
        // Create c1  
        CircleWithStaticMembers c1 = new CircleWithStaticMembers();  
  
        // Display c1 BEFORE c2 is created  
        System.out.println("\nAfter creating c1");  
        System.out.println("c1: radius (" + c1.radius +  
            ") and number of Circle objects (" +  
            c1.numberOfObjects + ")");  
  
        // Create c2  
        CircleWithStaticMembers c2 = new CircleWithStaticMembers(5);  
  
        // Modify c1  
        c1.radius = 9;  
  
        // Display c1 and c2 AFTER c2 was created  
        System.out.println("\nAfter creating c2 and modifying c1");  
        System.out.println("c1: radius (" + c1.radius +  
            ") and number of Circle objects (" +  
            c1.numberOfObjects + ")");  
        System.out.println("c2: radius (" + c2.radius +  
            ") and number of Circle objects (" +  
            c2.numberOfObjects + ")");  
    }  
}
```

Static variables and methods can be accessed without creating objects. Line 6 displays the number of objects, which is 0, since no objects have been created.



Static and Instance Methods

An instance method can invoke an instance or static method and access an instance or static data field. A static method can invoke a static method and access a static data field. However, a static method cannot invoke an instance method or access an instance data field, since static methods and static data fields don't belong to a particular object.



Examples

```
public class A {  
    int i = 5;  
    static int k = 2;  
  
    public static void main(String[] args) {  
        int j = i; // Wrong because i is an instance variable  
        m1(); // Wrong because m1() is an instance method  
    }  
  
    public void m1() {  
        // Correct since instance and static variables and methods  
        // can be used in an instance method  
        i = i + k + m2(i, k);  
    }  
  
    public static int m2(int i, int j) {  
        return (int)(Math.pow(i, j));  
    }  
}
```



Examples

```
public class A {  
    int i = 5;  
    static int k = 2;  
  
    public static void main(String[] args) {  
        A a = new A();  
        int j = a.i; // OK, a.i accesses the object's instance variable  
        a.m1(); // OK. a.m1() invokes the object's instance method  
    }  
  
    public void m1() {  
        i = i + k + m2(i, k);  
    }  
  
    public static int m2(int i, int j) {  
        return (int)(Math.pow(i, j));  
    }  
}
```

Visibility Modifiers and Accessor/Mutator Methods

- ❑ You can use the **public** visibility modifier for classes, methods, and data fields to denote that they can be accessed from **any other classes**.
- ❑ If no visibility modifier is used, then by default the classes, methods, and data fields are accessible by any class **in the same package**. This is known as *package-private* or *package-access*.
- ❑ Packages can be used to organize classes. To do so, you need to add the following line as the first statement in the program:

package packageName;

- ❑ If a class is defined without the package statement, it is said to be placed in the *default package*.



Visibility Modifiers and Accessor/Mutator Methods

By default, the class, variable, or method can be accessed by any class in the same package.

❑ `public`

The class, data, or method is visible to any class in any package.

❑ `private`

The data or methods can be accessed only by the declaring class.

Public getter (Accessor) and setter (Mutator) methods are used to read and modify **private** properties.



Examples

```
package p1;

public class C1 {
    public int x;
    int y;
    private int z;

    public void m1() {
    }
    void m2() {
    }
    private void m3() {
    }
}
```

```
package p1;

public class C2 {
    void aMethod() {
        C1 o = new C1();
        can access o.x;
        can access o.y;
        cannot access o.z;

        can invoke o.m1();
        can invoke o.m2();
        cannot invoke o.m3();
    }
}
```

```
package p2;

public class C3 {
    void aMethod() {
        C1 o = new C1();
        can access o.x;
        cannot access o.y;
        cannot access o.z;

        can invoke o.m1();
        cannot invoke o.m2();
        cannot invoke o.m3();
    }
}
```

- The private modifier restricts access to within a class,
- The default modifier restricts access to within a package,
- The public modifier enables unrestricted access.



Visibility Modifier of Classes

```
package p1;
```

```
class C1 {  
    ...  
}
```

```
package p1;
```

```
public class C2 {  
    can access C1  
}
```

```
package p2;
```

```
public class C3 {  
    cannot access C1;  
    can access C2;  
}
```

- The default modifier on a class restricts access to within a package
- The public modifier enables unrestricted access.



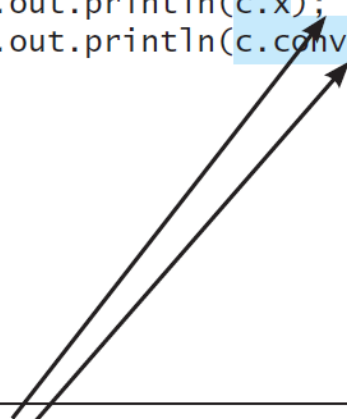
NOTE

- An object cannot access its private members, as shown in (b). It is OK, however, if the object is declared in its own class, as shown in (a).

```
public class C {  
    private boolean x;  
  
    public static void main(String[] args) {  
        C c = new C();  
        System.out.println(c.x);  
        System.out.println(c.convert());  
    }  
  
    private int convert() {  
        return x ? 1 : -1;  
    }  
}
```

(a) This is okay because object **c** is used inside the class **C**.

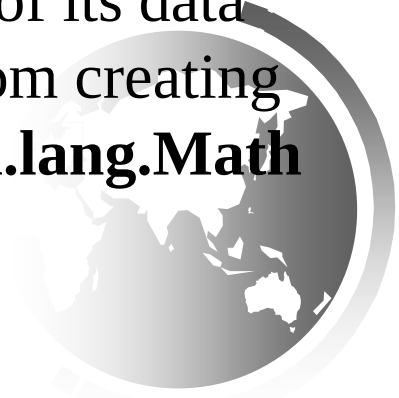
```
public class Test {  
    public static void main(String[] args) {  
        C c = new C();  
        System.out.println(c.x);  
        System.out.println(c.convert());  
    }  
}
```



(b) This is wrong because **x** and **convert** are private in class **C**.

NOTE

- The **private** modifier applies only to the members of a class. The **public** modifier can apply to a class or members of a class. Using the modifiers **public** and **private** on local variables would cause a compile error.
- In most cases, the constructor should be public. However, if you want to prohibit the user from creating an instance of a class, use a *private constructor*. For example, there is no reason to create an instance from the **Math** class, because all of its data fields and methods are static. To prevent the user from creating objects from the **Math** class, the constructor in **java.lang.Math** is defined as private.



Data Field Encapsulation

- ❑ *Making data fields private* protect data.
- ❑ *Making data fields private* helps to make code easy to maintain since the client programs cannot modify them.
- ❑ To prevent direct modifications of data fields, declaring the data fields private is known as *data field encapsulation*.
- ❑ To make a private data field accessible, provide a *getter* method to return its value. To enable a private data field to be updated, provide a *setter* method to set a new value. A getter method is also referred to as an *accessor* and a setter to a *mutator*.



Example of Data Field Encapsulation

The - sign indicates
a private modifier →

Circle
<code>-radius: double</code> <code>-<u>numberOfObjects</u>: int</code>
<code>+Circle()</code> <code>+Circle(radius: double)</code> <code>+getRadius(): double</code> <code>+setRadius(radius: double): void</code> <code>+<u>getNumberOfObjects</u>(): int</code> <code>+getArea(): double</code>

The radius of this circle (default: 1.0).
The number of circle objects created.

Constructs a default circle object.
Constructs a circle object with the specified radius.
Returns the radius of this circle.
Sets a new radius for this circle.
Returns the number of circle objects created.
Returns the area of this circle.



Example of Data Field Encapsulation

```
1 public class CircleWithPrivateDataFields {
2     /** The radius of the circle */
3     private double radius = 1;
4
5     /** The number of objects created */
6     private static int numberOfObjects = 0;
7
8     /** Construct a circle with radius 1 */
9     public CircleWithPrivateDataFields() {
10         numberOfObjects++;
11     }
12
13     /** Construct a circle with a specified radius */
14     public CircleWithPrivateDataFields(double newRadius) {
15         radius = newRadius;
16         numberOfObjects++;
```



Example of Data Field Encapsulation

```
17     }
18
19     /** Return radius */
20     public double getRadius() {
21         return radius;
22     }
23
24     /** Set a new radius */
25     public void setRadius(double newRadius) {
26         radius = (newRadius >= 0) ? newRadius : 0;
27     }
28
29     /** Return numberOfObjects */
30     public static int getNumberOfObjects() {
31         return numberOfObjects;
32     }
33
34     /** Return the area of this circle */
35     public double getArea() {
36         return radius * radius * Math.PI;
37     }
38 }
```



Example of Data Field Encapsulation

```
1 public class TestCircleWithPrivateDataFields {
2     /** Main method */
3     public static void main(String[] args) {
4         // Create a circle with radius 5.0
5         CircleWithPrivateDataFields myCircle =
6             new CircleWithPrivateDataFields(5.0);
7         System.out.println("The area of the circle of radius "
8             + myCircle.getRadius() + " is " + myCircle.getArea());
9
10        // Increase myCircle's radius by 10%
11        myCircle.setRadius(myCircle.getRadius() * 1.1);
12        System.out.println("The area of the circle of radius "
13            + myCircle.getRadius() + " is " + myCircle.getArea());
14
15        System.out.println("The number of objects created is "
16            + CircleWithPrivateDataFields.getNumberOfObjects());
17    }
18 }
```



Passing Objects to Methods

- ❑ Passing by value for primitive type value
(the value is passed to the parameter)
- ❑ Passing by value for reference type value
(the value is the reference to the object)



Passing Objects to Methods

```
public class Test {  
    public static void main(String[] args) {  
        // CircleWithPrivateDataFields is defined in Listing 9.8  
        CircleWithPrivateDataFields myCircle = new  
            CircleWithPrivateDataFields(5.0);  
        printCircle(myCircle);  
    }  
  
    public static void printCircle(CircleWithPrivateDataFields c) {  
        System.out.println("The area of the circle of radius "  
            + c.getRadius() + " is " + c.getArea());  
    }  
}
```



Passing a primitive type value and a reference value

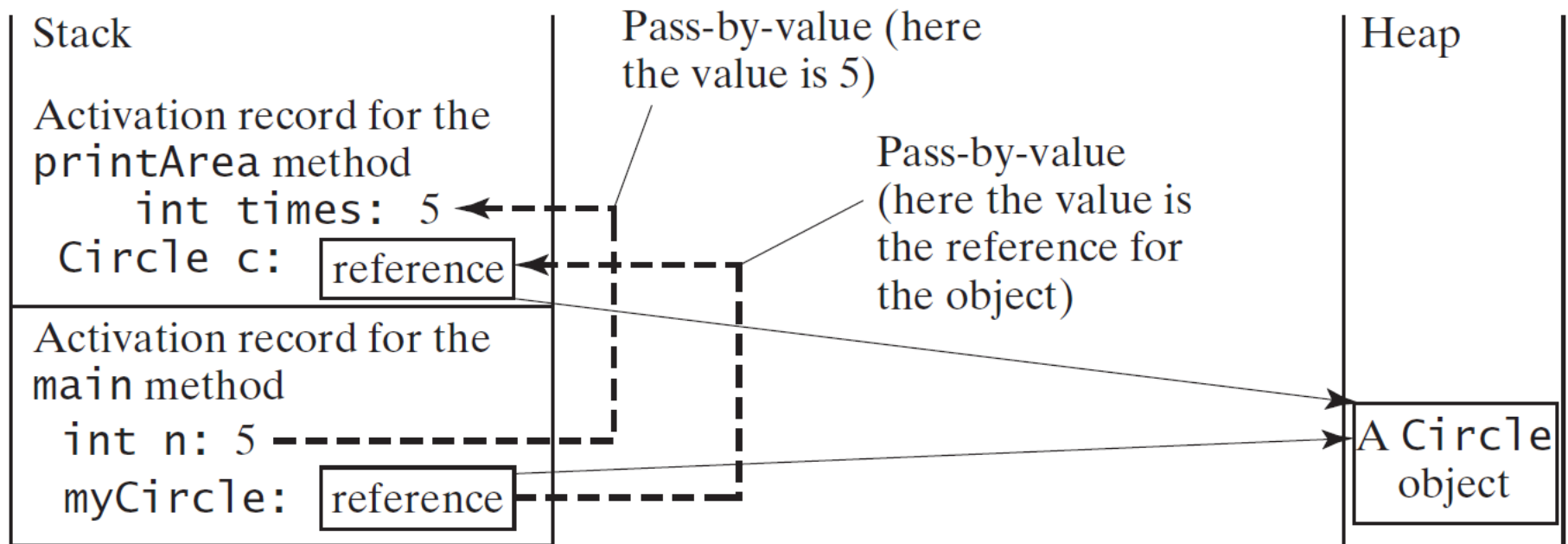
```
public class TestPassObject {  
    /** Main method */  
    public static void main(String[] args) {  
        // Create a Circle object with radius 1  
        CircleWithPrivateDataFields myCircle =  
            new CircleWithPrivateDataFields(1);  
  
        // Print areas for radius 1, 2, 3, 4, and 5.  
        int n = 5;  
        printAreas(myCircle, n);  
  
        // See myCircle.radius and times  
        System.out.println("\n" + "Radius is " + myCircle.getRadius());  
        System.out.println("n is " + n);  
    }  
  
    /** Print a table of areas for radius */  
    public static void printAreas(  
        CircleWithPrivateDataFields c, int times) {  
        System.out.println("Radius \t\tArea");  
        while (times >= 1) {  
            System.out.println(c.getRadius() + "\t\t" + c.getArea());  
            c.setRadius(c.getRadius() + 1);  
            times--;  
        }  
    }  
}
```

Passing a primitive type value and a reference value

```
Radius      Area
1.0         3.141592653589793
2.0         12.566370614359172
3.0         29.274333882308138
4.0         50.26548245743669
5.0         79.53981633974483
Radius is 6.0
n is 5
```

When passing an argument of a reference type, the reference of the object is passed. In this case, `c` contains a reference for the object that is also referenced via `myCircle`. Therefore, changing the properties of the object through `c` inside the `printAreas` method has the same effect as doing so outside the method through the variable `myCircle`. Pass-by-value on references can be best described semantically as *pass-by-sharing*; that is, the object referenced in the method is the same as the object being passed.

Passing Objects to Methods



Array of Objects

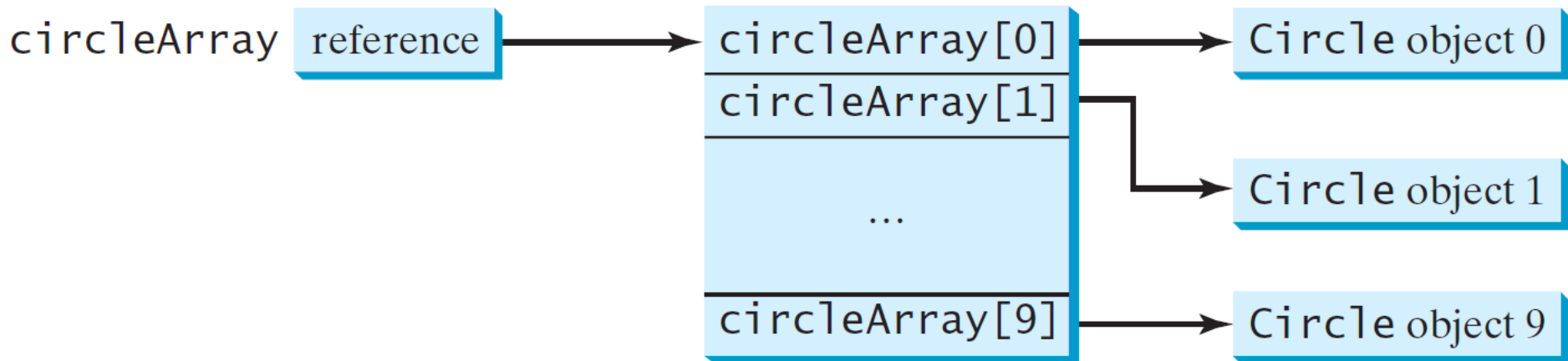
```
Circle[] circleArray = new Circle[10];
```

An array of objects is actually an *array of reference variables*. So invoking `circleArray[1].getArea()` involves two levels of referencing as shown in the next figure. `circleArray` references to the entire array. `circleArray[1]` references to a `Circle` object.



Array of Objects

```
Circle[] circleArray = new Circle[10];
```



Array of Objects

- To initialize **circleArray**, you can use a **for** loop
for (int i = 0; i < circleArray.length; i++) {
 circleArray[i] = new Circle();
}
- An array of objects is actually an *array of reference variables*.
- When an array of objects is created using the **new** operator, each element in the array is a reference variable with a default value of **null**.



Summarizing the areas of the circles

```
public class TotalArea {
    /** Main method */
    public static void main(String[] args) {
        // Declare circleArray
        CircleWithPrivateDataFields[] circleArray;

        // Create circleArray
        circleArray = createCircleArray();

        // Print circleArray and total areas of the circles
        printCircleArray(circleArray);
    }

    /** Create an array of Circle objects */
    public static CircleWithPrivateDataFields[] createCircleArray() {
        CircleWithPrivateDataFields[] circleArray =
            new CircleWithPrivateDataFields[5];

        for (int i = 0; i < circleArray.length; i++) {
            circleArray[i] =
                new CircleWithPrivateDataFields(Math.random() * 100);
        }

        // Return Circle array
        return circleArray;
    }
}
```



Summarizing the areas of the circles

```
/** Print an array of circles and their total area */
public static void printCircleArray(
    CircleWithPrivateDataFields[] circleArray) {
    System.out.printf("%-30s%-15s\n", "Radius", "Area");
    for (int i = 0; i < circleArray.length; i++) {
        System.out.printf("%-30f%-15f\n", circleArray[i].getRadius(),
            circleArray[i].getArea());
    }

    System.out.println("-----");

    // Compute and display the result
    System.out.printf("%-30s%-15f\n", "The total area of circles is",
        sum(circleArray) );
}
```



Summarizing the areas of the circles

```
/** Add circle areas */  
public static double sum(CircleWithPrivateDataFields[] circleArray) .  
    // Initialize sum  
    double sum = 0;  
  
    // Add areas to sum  
    for (int i = 0; i < circleArray.length; i++)  
        sum += circleArray[i].getArea();  
  
    return sum;  
}  
}
```



Immutable Objects

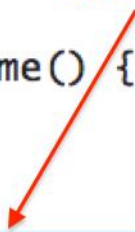
- Normally, you create an object and allow its contents to be changed later. However, occasionally it is desirable to create an object whose contents cannot be changed once the object has been created. We call such an object as *immutable object* and its class as *immutable class*.
- If a class is immutable, then all its data fields must be private and it cannot contain public setter methods for any data fields. **A class with all private data fields and no mutators is not necessarily immutable.**



Example

```
1 public class Student {
2     private int id;
3     private String name;
4     private java.util.Date dateCreated;
5
6     public Student(int ssn, String newName) {
7         id = ssn;
8         name = newName;
9         dateCreated = new java.util.Date();
10    }
11
12    public int getId() {
13        return id;
14    }
15
16    public String getName() {
17        return name;
18    }
19
20    public java.util.Date getDateCreated() {
21        return dateCreated;
22    }
23 }
```

Public method returning a reference!!!



Example

As shown in the following code, the data field **dateCreated** is returned using the **getDateCreated()** method. This is a reference to a **Date** object. Through this reference, the content for **dateCreated** can be changed.

```
public class Test {  
    public static void main(String[] args) {  
        Student student = new Student(111223333, "John");  
        java.util.Date dateCreated = student.getDateCreated();  
        dateCreated.setTime(200000); // Now dateCreated field is changed!  
    }  
}
```

For a class to be immutable, it must meet the following requirements:

- All data fields must be private.
- There can't be any mutator methods for data fields.
- No accessor methods can return a reference to a data field that is mutable.

Scope of Variables

- ❑ The scope of instance and static variables is the entire class. They can be declared anywhere inside a class.
- ❑ The scope of a local variable (i.e. a variable defined in a method) starts from its declaration and continues to the end of the block that contains the variable. A local variable must be initialized explicitly before it can be used.



Scope of Variables

```
public class Circle {  
    public double findArea() {  
        return radius * radius * Math.PI;  
    }  
  
    private double radius = 1;  
}
```

- (a) The variable **radius** and method **findArea()** can be declared in any order.



Scope of Variables

- If a local variable has the same name as a class's variable, the local variable takes precedence and the class's variable with the same name is *hidden*.

```
public class F {  
    private int x = 0; // Instance variable  
    private int y = 0;  
  
    public F() {  
    }  
  
    public void p() {  
        int x = 1; // Local variable  
        System.out.println("x = " + x);  
        System.out.println("y = " + y);  
    }  
}
```



The this Keyword

- ❑ The this keyword is the name of a reference that refers to an object itself. One common use of the this keyword is reference a class's *hidden data fields*.
- ❑ Another common use of the this keyword to enable a constructor to invoke another constructor of the same class.



The this Keyword

```
public class Circle {  
    private double radius;  
  
    ...  
  
    public double getArea() {  
        return this.radius * this.radius * Math.PI;  
    }  
  
    public String toString() {  
        return "radius: " + this.radius  
            + "area: " + this.getArea() ;  
    }  
}
```

Equivalent

```
public class Circle {  
    private double radius;  
  
    ...  
  
    public double getArea() {  
        return radius * radius * Math.PI;  
    }  
  
    public String toString() {  
        return "radius: " + radius  
            + "area: " + getArea() ;  
    }  
}
```



Reference the Hidden Data Fields

The **this** keyword can be used to reference a class's *hidden data fields*. For example, a data-field name is often used as the parameter name in a setter method for the data field. In this case, the data field is hidden in the setter method. You need to reference the hidden data-field name in the method in order to set a new value to it. A hidden static variable can be accessed using the keyword **this**.

```
public class F {  
    private int i = 5;  
    private static double k = 0;  
  
    public void setI(int i) {  
        this.i = i;  
    }  
  
    public static void setK(double k) {  
        F.k = k;  
    }  
  
    // Other methods omitted  
}
```



Reference the Hidden Data Fields

The **this** keyword gives us a way to reference the object that invokes an instance method. To invoke **f1.setI(10)**, **this.i = i** is executed, which assigns the value of parameter **i** to the data field **i** of this calling object **f1**. The keyword **this** refers to the object that invokes the instance method **setI**, as shown below:

Suppose that **f1** and **f2** are two objects of **F**.

Invoking **f1.setI(10)** is to execute
this.i = 10, where **this** refers **f1**

Invoking **f2.setI(45)** is to execute
this.i = 45, where **this** refers **f2**

Invoking **F.setK(33)** is to execute
F.k = 33. **setK** is a static method



Calling Overloaded Constructor

The **this** keyword can be used to invoke another constructor of the same class. For example, you can rewrite the **Circle** class as follows:

```
public class Circle {  
    private double radius;  
  
    public Circle(double radius) {  
        this.radius = radius;  
    }  
  
    public Circle() {  
        this(1.0);  
    }  
  
    ...  
}
```

The **this** keyword is used to reference the hidden data field **radius** of the object being constructed.

The **this** keyword is used to invoke another constructor.

